

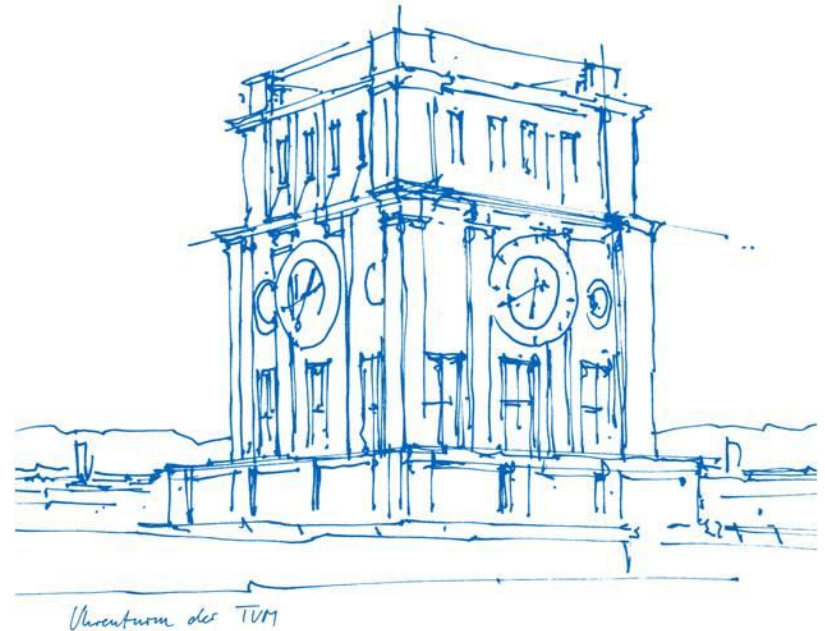
Basics of FIM: Statistical Learning

5: Decision Trees

Univ.-Prof. Dr-Ing. habil. Niklas Kühl

www.niklas.xyz

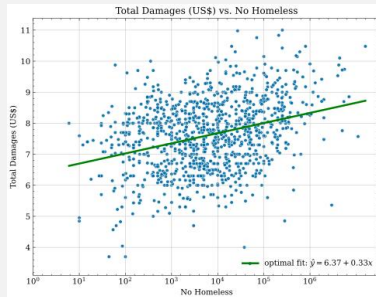
Chair for Information Systems and Human-centric
Artificial Intelligence
University of Bayreuth
TUM School of Management



All previous methods made strong assumptions.

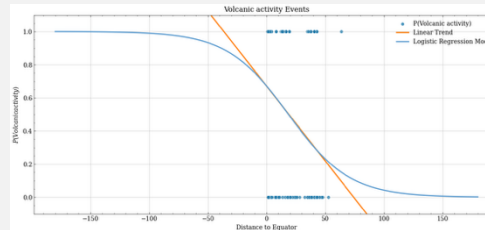
Linear Regression

$$Y = X\beta + \varepsilon$$



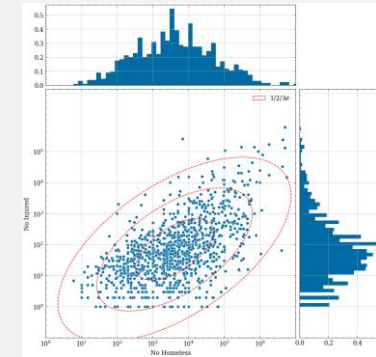
Logistic Regression

$$P(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}$$



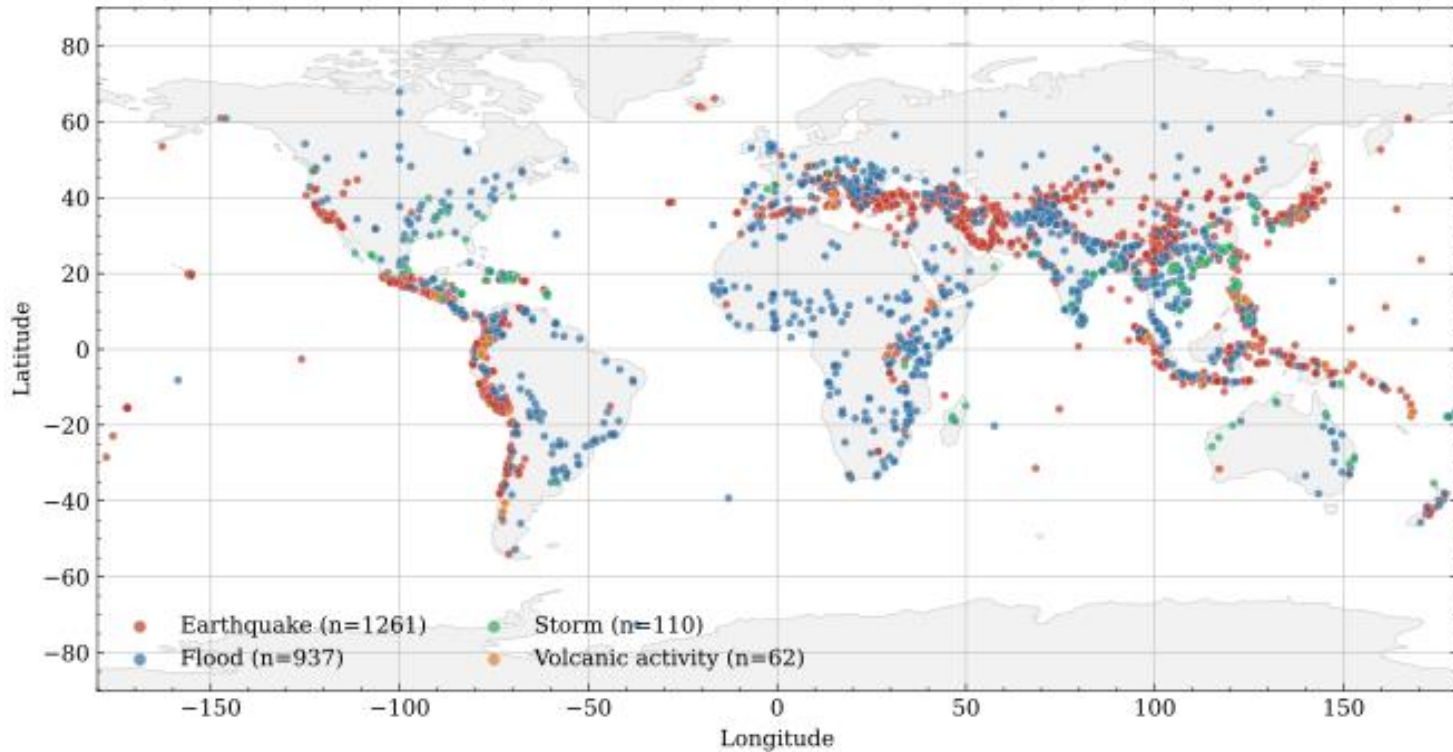
Discriminant analysis

$$P(Y = k | X = x) = \frac{\pi_k \times f_k(x)}{\sum_{l=1}^K \pi_l f_l(x)}$$

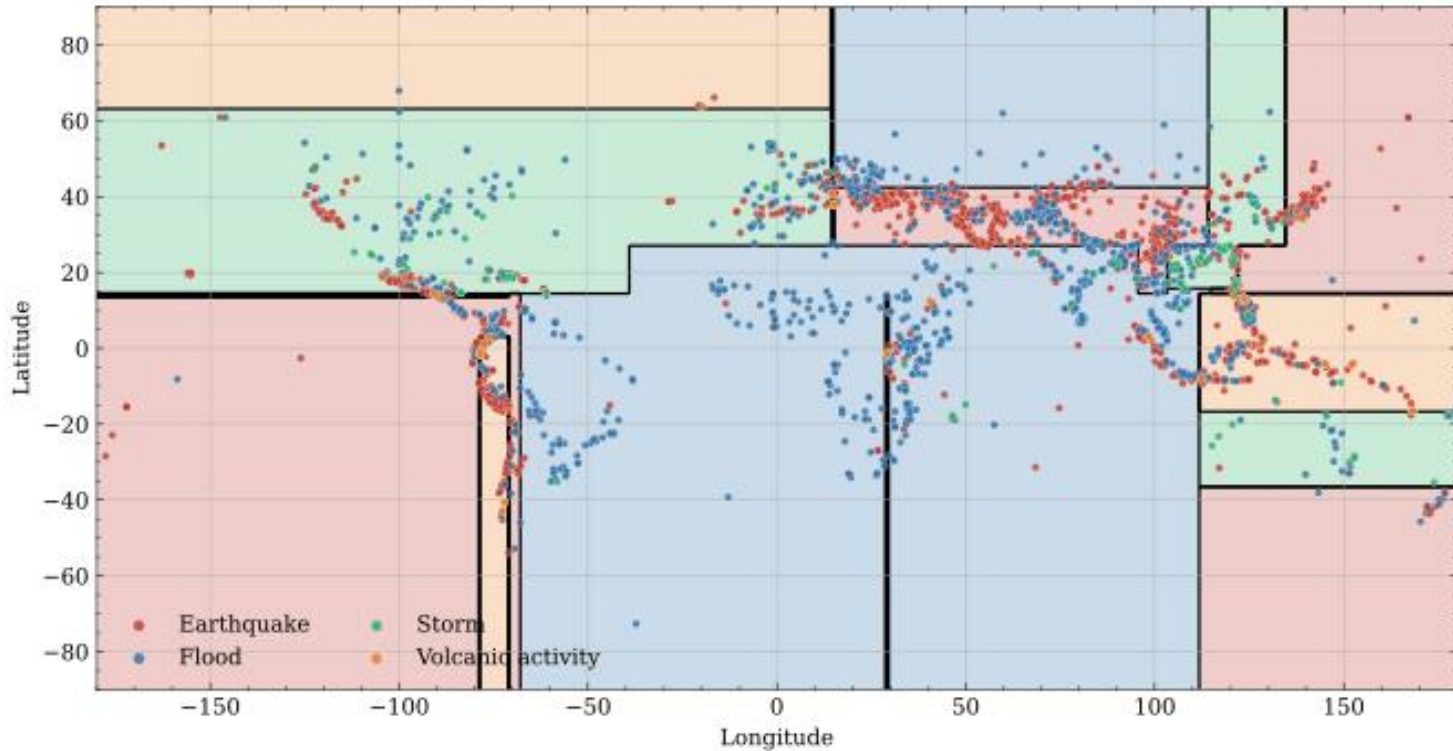


Which assumption do all of these methods have in common?

Disaster type is governed by the region of the map — not a global function.



We assign different regions across our features.

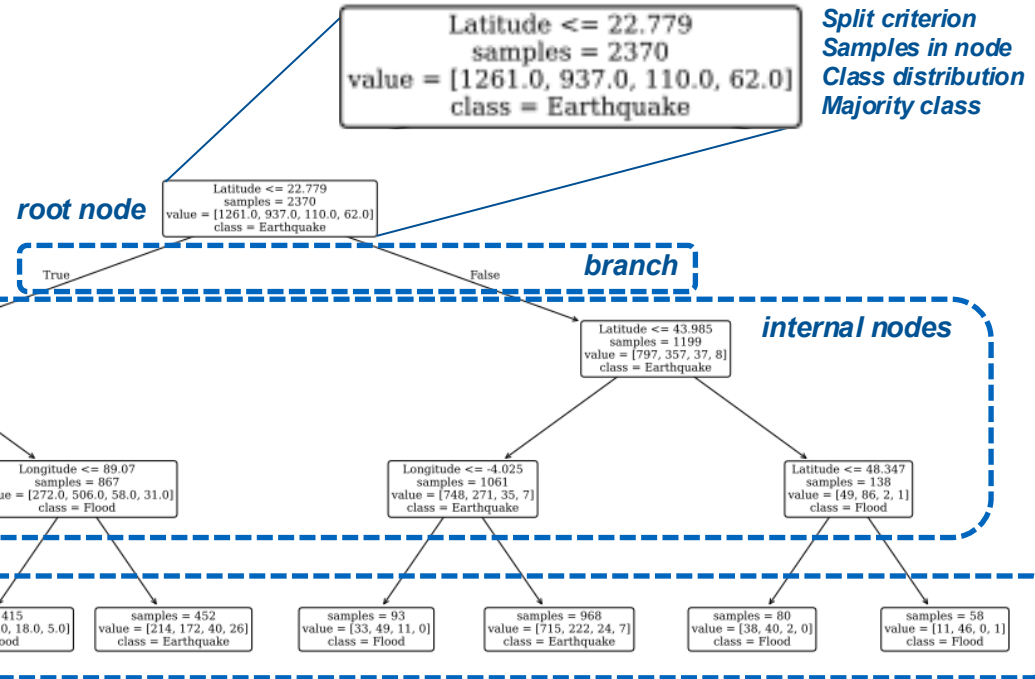
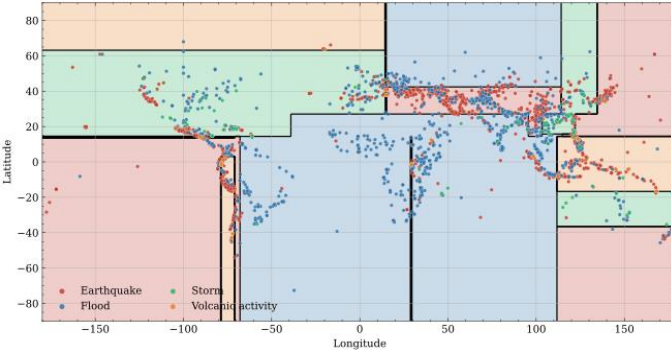


- Learning Decision Trees
- Pruning Decision Trees
- Popular Algorithms
- Tree Ensembles

- Learning Decision Trees
- Pruning Decision Trees
- Popular Algorithms
- Tree Ensembles

Those regions are specified by simple yes or no questions.

Tree-based methods segment the predictor space into a number of simple regions.



For a new instance X we follow the splits and assign for a leaf node R_j :

- e.g. $X = [X_{longitude}, X_{latitude}] = [15, 32]$



How do we decide on the split feature and the split value?

Greedy, top-down recursive binary splitting

- Consider every feature X_j and every threshold s .
- Score a split by the weighted **(im-)purity** of its two children compared to the parent node, which we call information gain (**IG**).
- Keep the best split; recurse inside each child region.

```
Latitude <= 22.779
samples = 2370
value = [1261.0, 937.0, 110.0, 62.0]
class = Earthquake
```

$$(j^*, s^*) = \arg \max_{j, s} IG$$

$$= \arg \max_{j, s} I(\text{parent}) - \sum_{C \in \{L, R\}} \frac{N_C}{N} I(\text{child}_C)$$

We can measure the **(im-)purity** in a Region R_m by:

Misclassification error

$$E_m = 1 - \max_k \hat{p}_{mk}$$

Gini index

$$G_m = \sum_k \hat{p}_{mk} (1 - \hat{p}_{mk}) = 1 - \sum_k \hat{p}_{mk}^2$$

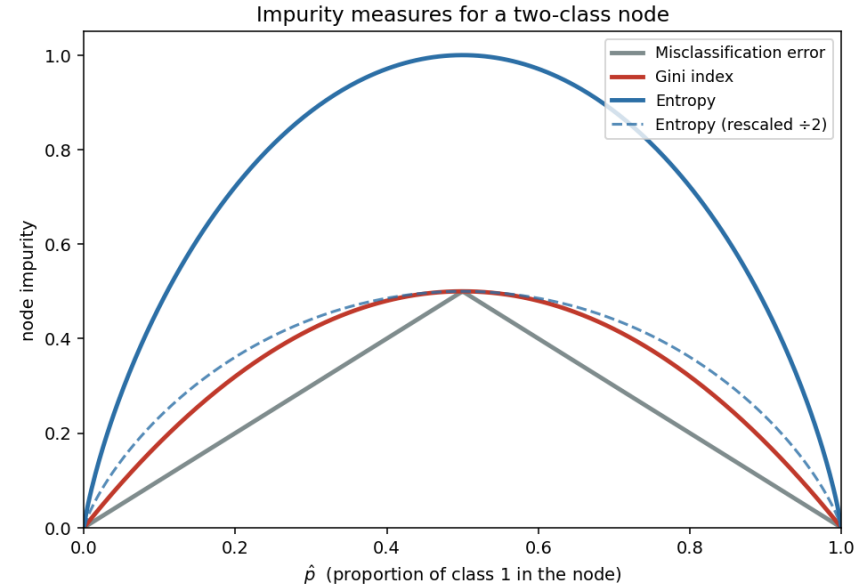
Entropy

$$H_m = - \sum_k \hat{p}_{mk} \log_2 \hat{p}_{mk}$$

How do we decide on the split feature and the split value?

Greedy, top-down recursive binary splitting

- Consider every feature X_j and every threshold s .
- Score a split by the weighted **(im-)purity** of its two children compared to the parent node, which we call information gain (**IG**).
- Keep the best split; recurse inside each child region.



We can measure the **(im-)purity** in a Region R_m by:

Misclassification error

$$E_m = 1 - \max_k \hat{p}_{mk}$$

Gini index

$$G_m = \sum_k \hat{p}_{mk} (1 - \hat{p}_{mk}) = 1 - \sum_k \hat{p}_{mk}^2$$

Entropy

$$H_m = - \sum_k \hat{p}_{mk} \log_2 \hat{p}_{mk}$$

Worked example on the disaster data — Earthquake vs. Flood

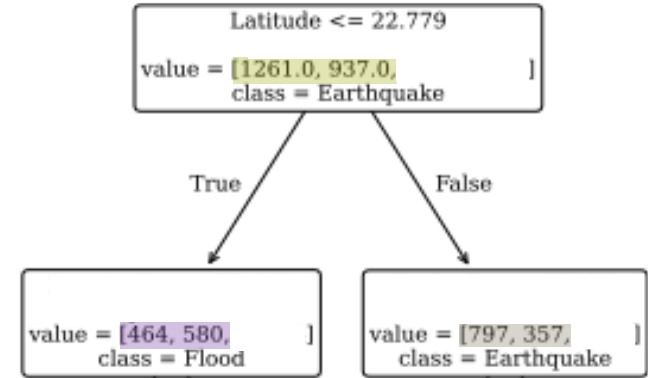
Greedy*, top-down recursive binary splitting

$$(j^*, s^*) = \arg \max_{j, s} IG$$

$$= \arg \max_{j, s} I(\text{parent}) \sum_{C \in \{L, R\}} \frac{N_C}{N} I(\text{child}_C)$$

Gini index

$$G_m = \sum_k \hat{p}_{mk} (1 - \hat{p}_{mk}) = 1 - \sum_k \hat{p}_{mk}^2$$



Parent node: 1261 EQ / 937 Flood → **H = 0.984**

Left ($\leq 22.75^\circ$, $n = 1044$): 464 EQ / 580 Flood → **H = 0.991**

Right ($> 22.75^\circ$, $n = 1154$): 797 EQ / 357 Flood → **H = 0.892**

$$IG = 0.984 - (1044/2198 \cdot 0.991 + 1154/2198 \cdot 0.892) = 0.984 - 0.939 = \mathbf{0.045}$$

→ Even the **best** single split barely improves purity — one question is not enough, which is why the tree keeps splitting (and why we then need to control its depth).

Finding the globally optimal tree is NP-hard.

Greedy*, top-down recursive binary splitting

- Consider every feature X_j and every threshold s .
- Score a split by the weighted **(im-)purity** of its two children compared to the parent node, which we call information gain (**IG**).
- Keep the best split; recurse inside each child region.



So how good is this algorithm?

$$(j^*, s^*) = \arg \max_{j, s} IG$$

$$= \arg \max_{j, s} I(\text{parent}) \sum_{C \in \{L, R\}} \frac{N_C}{N} I(\text{child}_C)$$

Gini index

$$G_m = \sum_k \hat{p}_{mk} (1 - \hat{p}_{mk}) = 1 - \sum_k \hat{p}_{mk}^2$$

* Greedy ≠ globally optimal

The algorithm fixes the best split **now**, without looking ahead. A weak split that would enable a great one below it is never taken. **Finding the globally optimal tree is NP-hard.** So we accept a fast, locally optimal greedy search.

Recursion stops at pure leaf nodes.

Greedy, top-down recursive binary splitting

- Consider every feature X_j and every threshold s .
- Score a split by the weighted **(im-)purity** of its two children compared to the parent node, which we call information gain (**IG**).
- Keep the best split; recurse inside each child region.

$$(j^*, s^*) = \arg \max_{j, s} \text{IG}$$

$$= \arg \max_{j, s} I(\text{parent}) \sum_{C \in \{L, R\}} \frac{N_C}{N} I(\text{child}_C)$$

Gini index

$$G_m = \sum_k \hat{p}_{mk} (1 - \hat{p}_{mk}) = 1 - \sum_k \hat{p}_{mk}^2$$



And when does the algorithm stop?

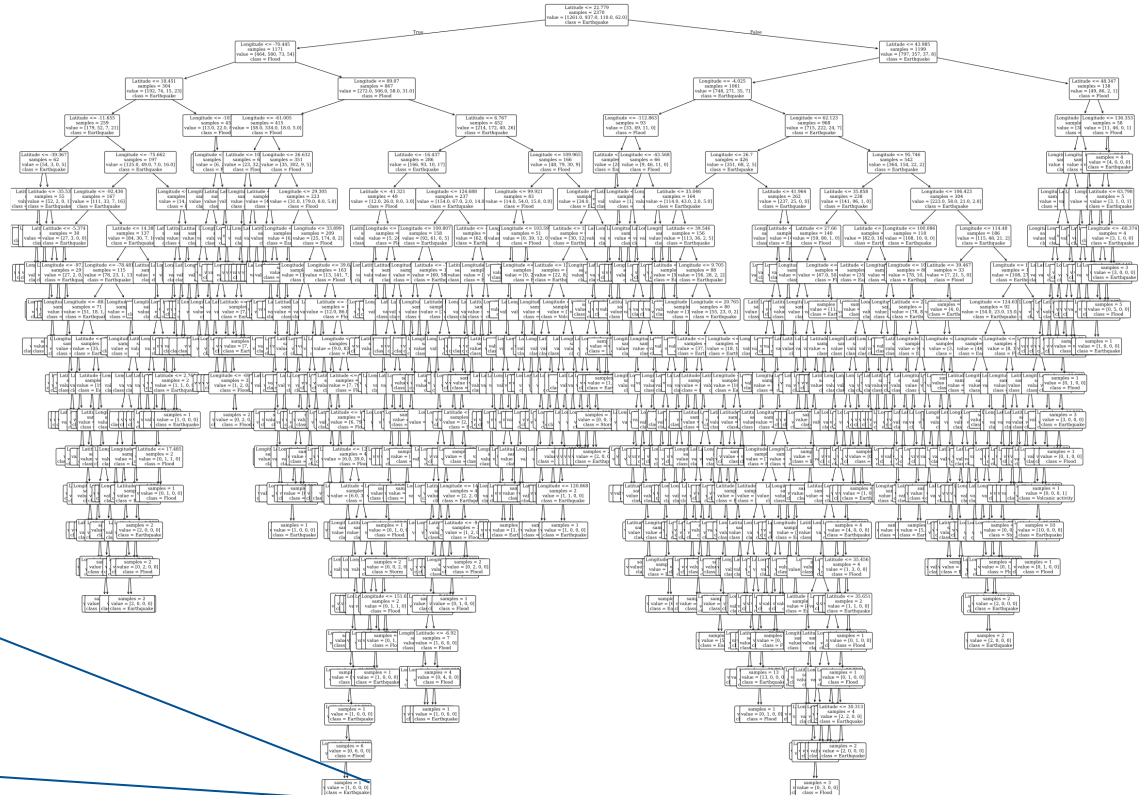
Recursion stops at pure leaf nodes.



And when does the algorithm stop?

* Recursion stops at pure leaf nodes

Without stop criterion, the recursive algorithm continues splitting nodes, till all leafes hold a single class/value.



samples = 1
value = [1, 0, 0, 0]
class = Earthquake

Recursion stops at pure leaf nodes.

?

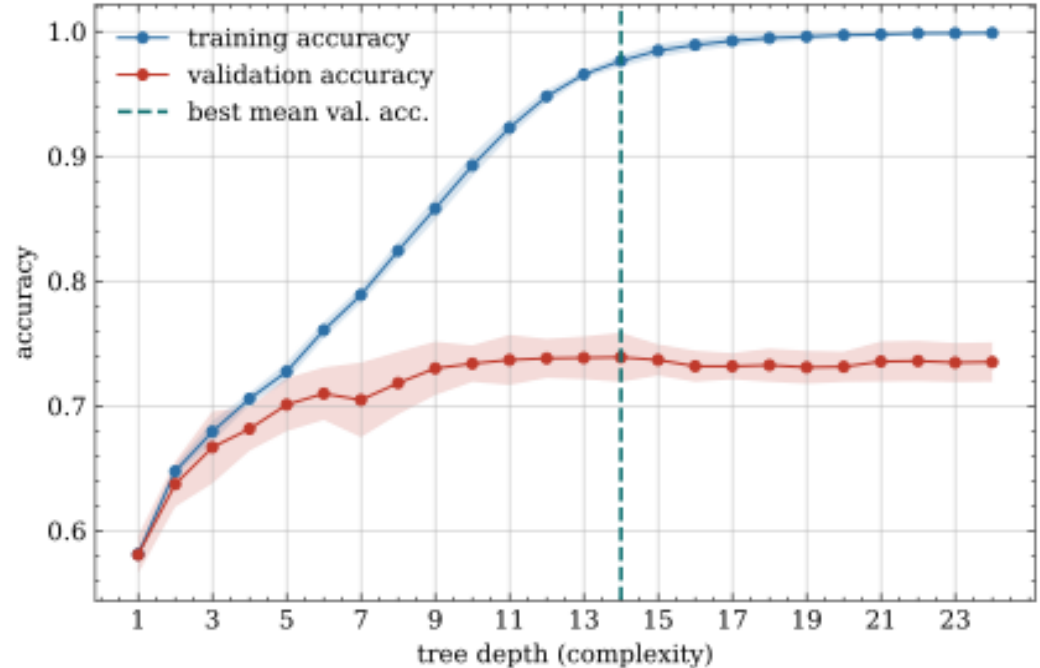
And when does the algorithm stop?

* Recursion stops at pure leaf nodes

Without stop criterion, the recursive algorithm continues splitting nodes, till all leafes hold a single class/value.

- Pure leaves: training accuracy $\rightarrow$ 100%
- validation accuracy plateaus (overfitting)

$\rightarrow$ We need to control the tree's complexity so it generalizes to new instances



- Learning Decision Trees
- Pruning Decision Trees
- Popular Algorithms
- Tree Ensembles

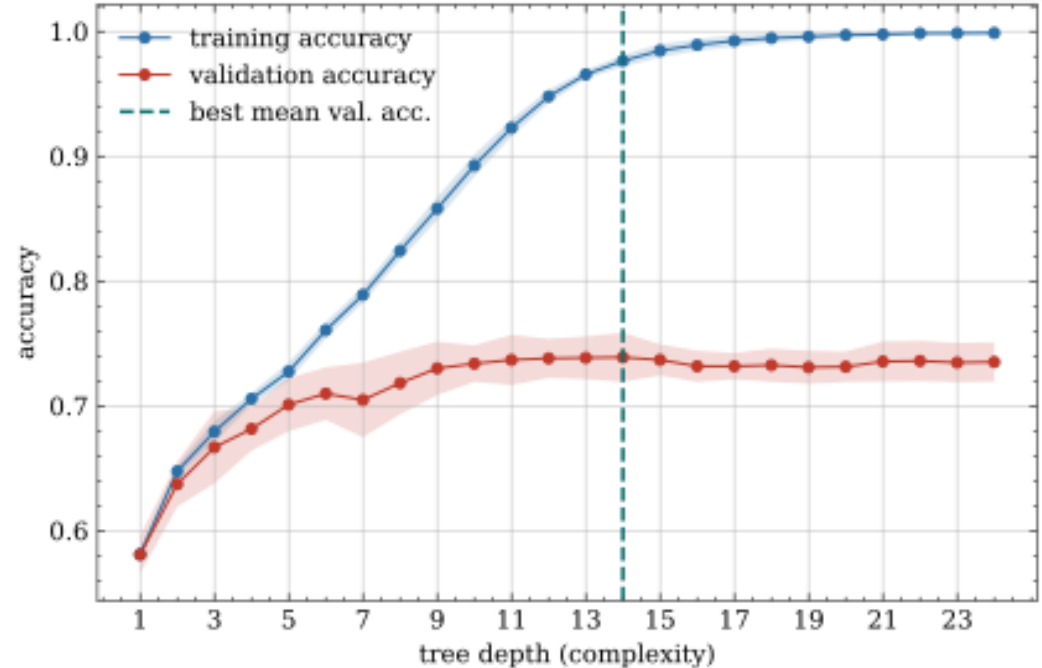
→ We need to control the tree's complexity so it generalizes to new instances

Pre-pruning (stop early)

Stop growing **before** the gap opens — set rules that halt splitting.

Post-pruning (grow, then cut back)

Grow the full tree, then **cut it back** to the sweet spot where validation accuracy peaks.



→ Same goal (land near the dashed line) reached from opposite directions.

Pre-pruning (stop early): set rules that halt splitting

Depth & size limits

```
max_depth = None
```

```
max_leaf_nodes = None
```

Minimum-sample rules

```
min_samples_split = 2
```

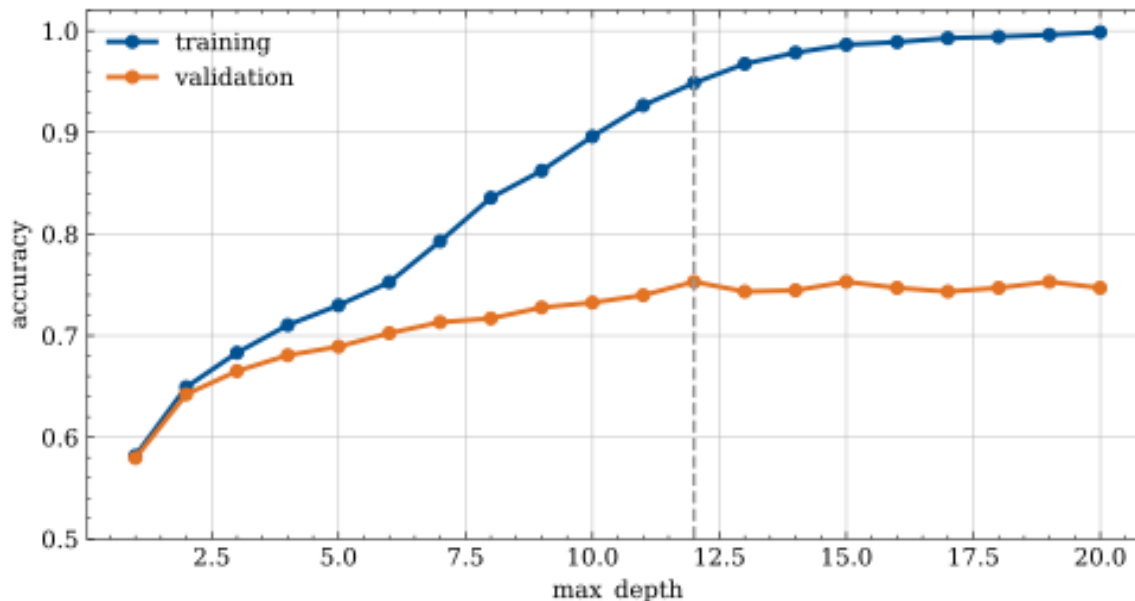
```
min_samples_leaf = 1
```

```
min_weight_fraction_leaf = 0.0
```

Minimum-gain rule

```
min_impurity_decrease = 0.0
```

max_depth: limit the maximum number of splits the algorithm can do in each branch



Pre-pruning (stop early): set rules that halt splitting

Depth & size limits

```
max_depth = None
```

```
max_leaf_nodes = None
```

Minimum-sample rules

```
min_samples_split = 2
```

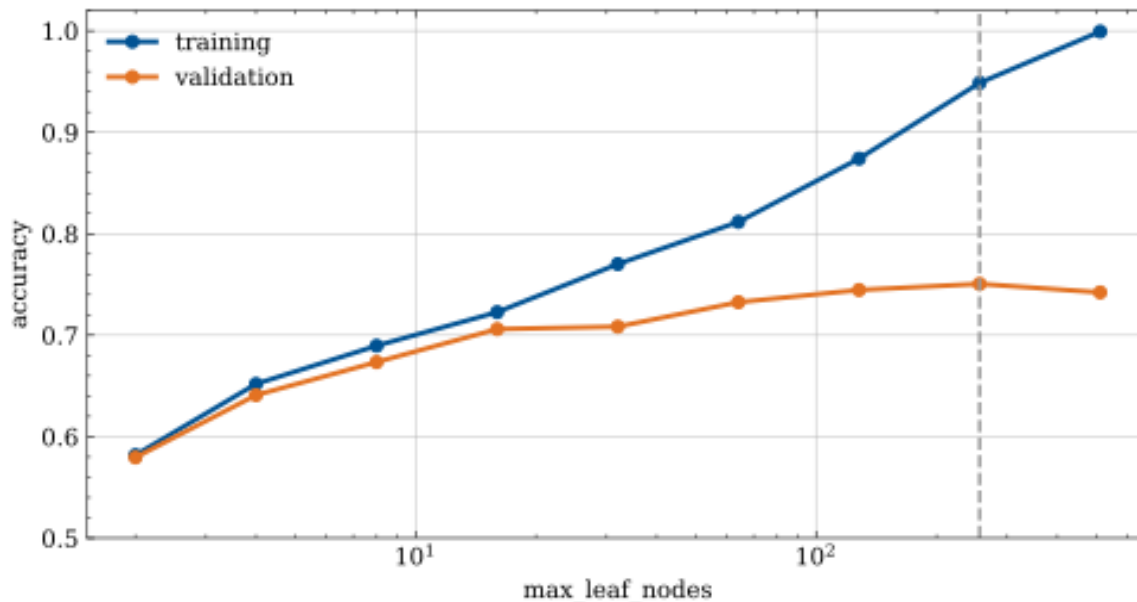
```
min_samples_leaf = 1
```

```
min_weight_fraction_leaf = 0.0
```

Minimum-gain rule

```
min_impurity_decrease = 0.0
```

max_leaf_nodes: cap the total number of leaf regions the tree may create



Pre-pruning (stop early): set rules that halt splitting

Depth & size limits

```
max_depth = None
```

```
max_leaf_nodes = None
```

Minimum-sample rules

```
min_samples_split = 2
```

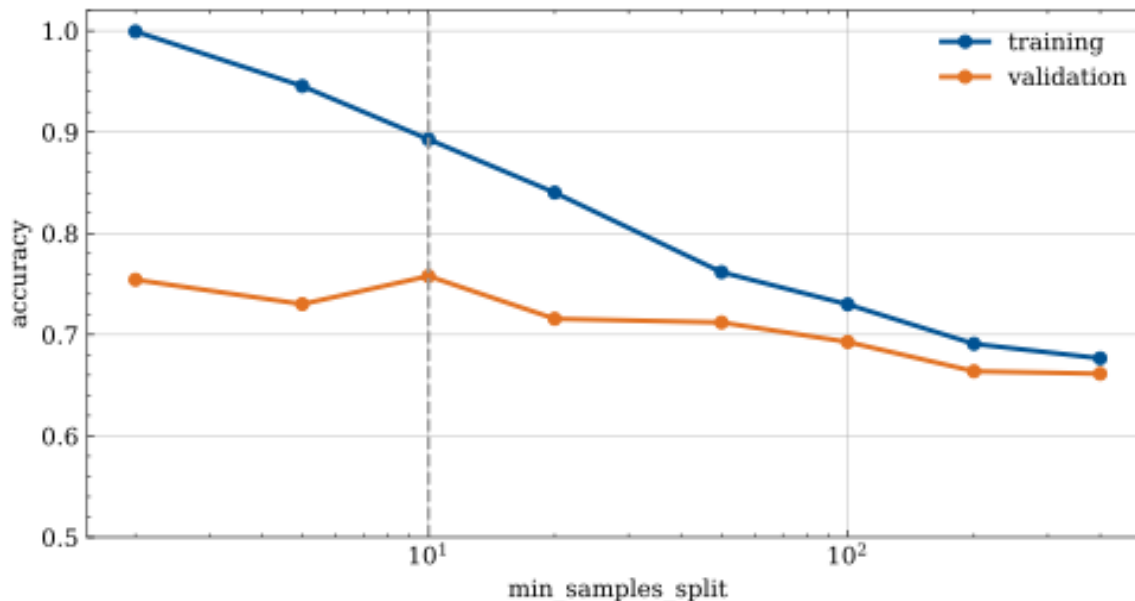
```
min_samples_leaf = 1
```

```
min_weight_fraction_leaf = 0.0
```

Minimum-gain rule

```
min_impurity_decrease = 0.0
```

min_samples_split: only split a node that still holds at least this many samples



Pre-pruning (stop early): set rules that halt splitting

Depth & size limits

```
max_depth = None
```

```
max_leaf_nodes = None
```

Minimum-sample rules

```
min_samples_split = 2
```

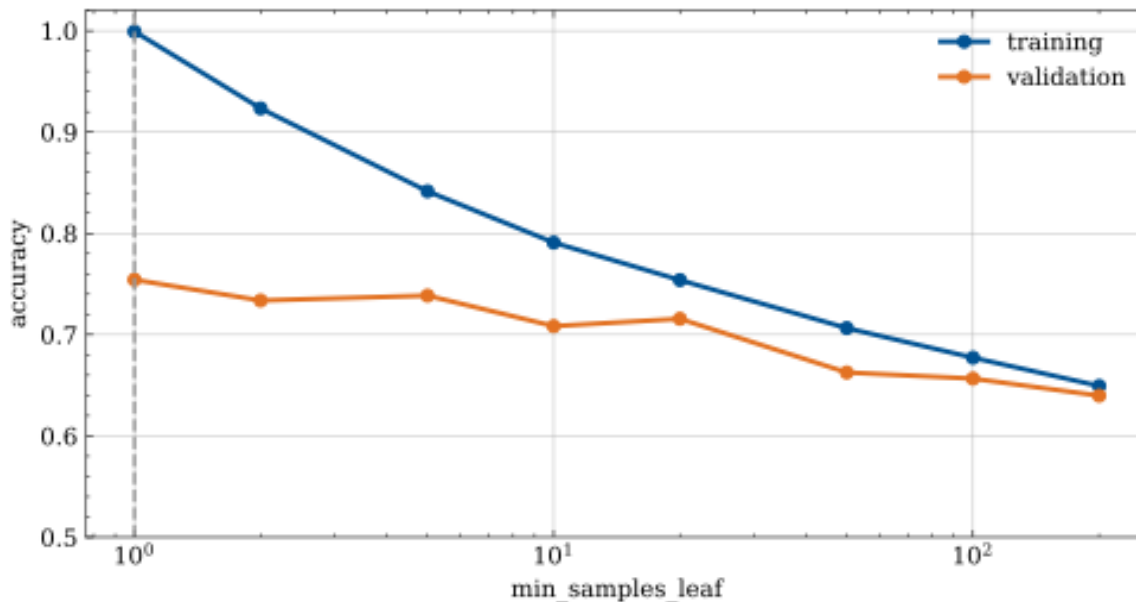
```
min_samples_leaf = 1
```

```
min_weight_fraction_leaf = 0.0
```

Minimum-gain rule

```
min_impurity_decrease = 0.0
```

min_samples_leaf: require every leaf to keep at least this many samples after a split



Pre-pruning (stop early): set rules that halt splitting

Depth & size limits

```
max_depth = None
```

```
max_leaf_nodes = None
```

Minimum-sample rules

```
min_samples_split = 2
```

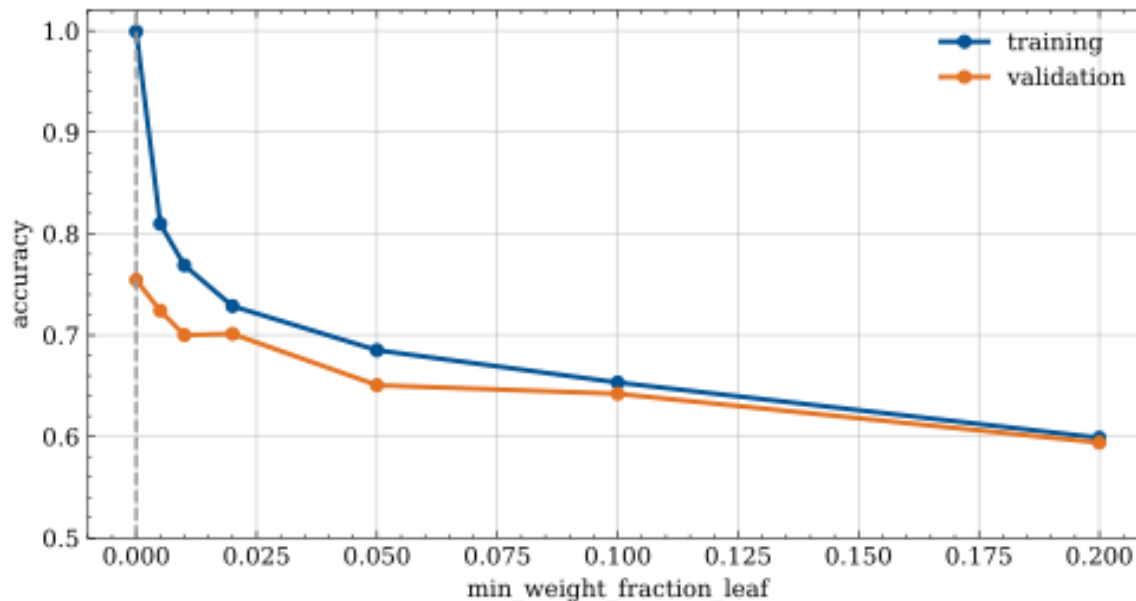
```
min_samples_leaf = 1
```

```
min_weight_fraction_leaf = 0.0
```

Minimum-gain rule

```
min_impurity_decrease = 0.0
```

min_weight_fraction_leaf: like min_samples_leaf, but as a fraction of the total (weighted) samples



Pre-pruning (stop early): set rules that halt splitting

Depth & size limits

```
max_depth = None
```

```
max_leaf_nodes = None
```

Minimum-sample rules

```
min_samples_split = 2
```

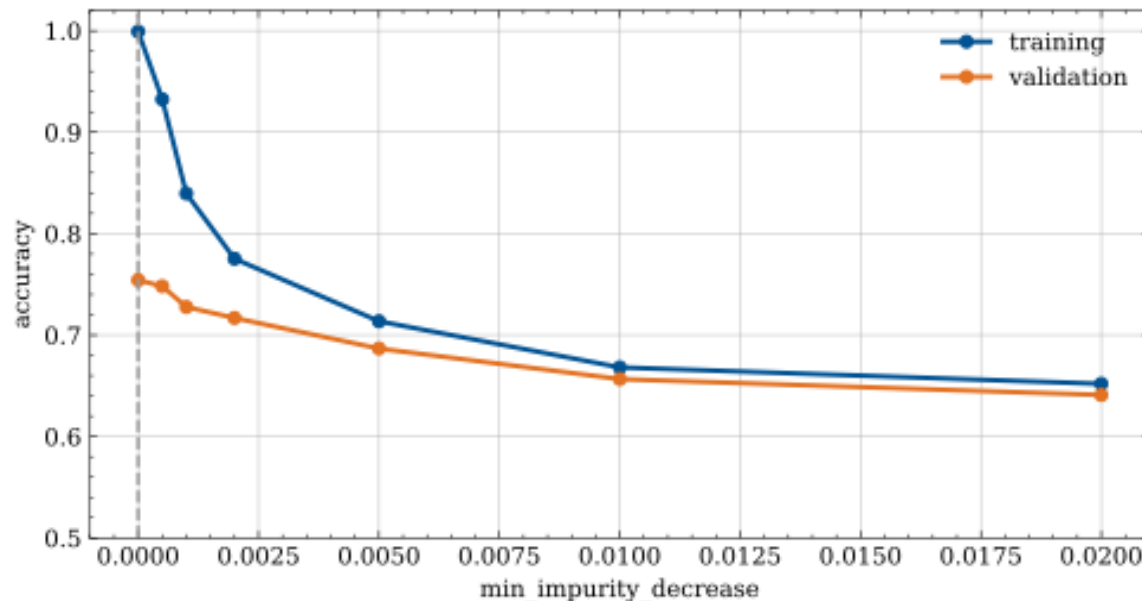
```
min_samples_leaf = 1
```

```
min_weight_fraction_leaf = 0.0
```

Minimum-gain rule

```
min_impurity_decrease = 0.0
```

min_impurity_decrease: only allow a split if it reduces impurity by at least this amount



Post-pruning: grow the tree fully, then cut back the weakest links

- Pre-pruning stops greedily; post-pruning grows the **full tree first**, then removes subtrees.
- Make every leaf “**pay**” a **penalty α** : a subtree is kept only if its gain in performance outweighs the cost of its extra leaves:

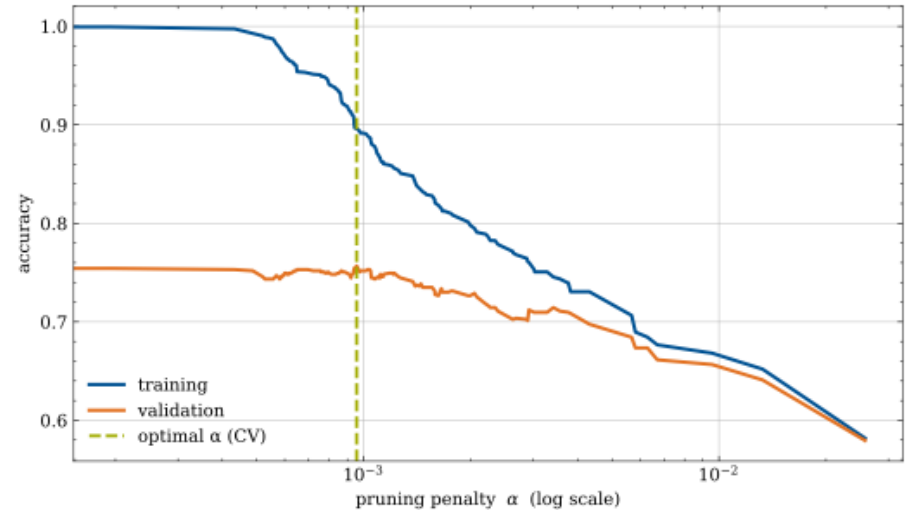
$$R_{\alpha}(T) = R(T) + \alpha|T|$$

T : a (sub-)tree

$R(T)$: training error — the **fit**

$\alpha|T|$: penalty $\times$ number of leaves — the **size**

- Raising α removes the **weakest link** (smallest error rise per leaf) first $\rightarrow$ a nested sequence of smaller trees.



- $\alpha = 0 \rightarrow$ full tree; $\alpha \rightarrow \infty \rightarrow$ just the root.
- Choose α by **cross-validation** — the value with the best validation accuracy.

At the optimal α the tree shrinks from **390** $\rightarrow$ **157 leaves** at the **same accuracy**.

- Learning Decision Trees
- Pruning Decision Trees
- Popular Algorithms
- Tree Ensembles

Tree Construction – Iterative Dichotomiser (ID3)

Algorithm 1 ID3 Algorithm

```

1: function ID3( $\mathcal{S} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ , Features  $\mathcal{F} = \{1, \dots, F\}$ )
2:   if  $y_i = c$  for all  $i = 1, \dots, n$  then
3:     return a leaf node with class  $c$ 
4:   end if
5:   if  $\mathcal{F} = \emptyset$  then
6:     return a leaf node with class  $\text{argmax}_c \sum_i \mathbb{I}_{\{y_i=c\}}$ 
7:   else
8:     Select the best feature  $j \in \mathcal{F}$  by information gain
9:     Create a decision tree node with  $j$ 
10:    for each value  $v \in \{x_{ij} : (\mathbf{x}_i, y_i) \in \mathcal{S}\}$  do
11:      Add a branch corresponding to  $j = v$ 
12:      Call ID3( $\{(\mathbf{x}_i, y_i) : (\mathbf{x}_i, y_i) \in \mathcal{S}, x_{ij} = v\}, \mathcal{F} \setminus \{j\}$ )
13:    end for
14:  end if
15:  return the decision tree
16: end function

```

Every feature corresponds to a dimension of the data vector (note: $n = N$ in the root node):

$$\mathbf{x}_i \in \mathbb{R}^F, i = 1, \dots, n$$

If no more features are left to split on, we add a leaf node representing a majority vote

We select the best split by information gain and create one branch for every realization of the identified feature

We call the algorithm on the resulting tree as long as the termination criteria are not met

First introduced by Quinlan (1986), ID3 is a decision tree construction algorithm that uses entropy as a measure to select the best feature for splitting at each node, aiming to maximize information gain during the tree construction.

Quinlan, J. R. 1986. Induction of Decision Trees. Mach. Learn. 1, 1 (Mar. 1986), 81-106.

Tree Construction – C4.5

Algorithm 1 C4.5 Algorithm

```

1: function C4.5( $\mathcal{S} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ , Features  $\mathcal{F} = \{1, \dots, F\}$ )
2:   if  $y_i = c$  for all  $i = 1, \dots, n$  then
3:     return a leaf node with class  $c$ 
4:   end if
5:   if  $\mathcal{F} = \emptyset$  then
6:     return a leaf node with class  $\operatorname{argmax}_c \sum_i \mathbb{I}_{\{y_i=c\}}$ 
7:   else
8:     Select the best feature  $j \in \mathcal{F}$  by gain ratio
9:     Create a decision tree node with  $j$ 
10:    for each value  $v \in \{x_{ij} : (\mathbf{x}_i, y_i) \in \mathcal{S}\}$  do
11:      Add a branch corresponding to  $j = v$ 
12:      Call C4.5( $\{(\mathbf{x}_i, y_i) : (\mathbf{x}_i, y_i) \in \mathcal{S}, x_{ij} = v\}, \mathcal{F} \setminus \{j\}$ )
13:    end for
14:    if pruning condition is met then:
15:      return a leaf node with class  $\operatorname{argmax}_c \sum_i \mathbb{I}_{\{y_i=c\}}$ 
16:    end if
17:  end if
18:  return the decision tree
19: end function

```

C4.5 is based on gain ratio instead of information gain (as in ID3). Thus, C4.5 is less biased towards branching based on features with many heterogenous realizations

error-based pessimistic pruning: No separate validation set but take node error rate as upper limit of a confidence interval under normal approximation of a binomial experiment. Walk the tree from the leaves upward. At each internal node, compare two pessimistic error totals: sum of the errors over all its leaves vs. error of the node above.

C4.5 is an improvement upon ID3 because it uses gain ratio as criteria for feature selection, and it incorporates a pruning step to avoid overfitting during the tree construction.

Quinlan, J. R. C4.5: Programs for Machine Learning. Morgan Kaufmann Publishers, 1993.

Tree Construction – CART

Algorithm 1 CART Algorithm

```

1: function CART( $\mathcal{S} = \{(x_i, y_i)\}_{i=1}^n$ , Features  $\mathcal{F} = \{1, \dots, F\}$ )
2:   if  $y_i = c$  for all  $i = 1, \dots, n$  then
3:     return a leaf node with class  $c$ 
4:   end if
5:   if  $\mathcal{F} = \emptyset$  then
6:     return a leaf node with class  $\operatorname{argmax}_c \sum_i \mathbb{I}_{\{y_i=c\}}$ 
7:   else
8:     Select the best feature  $j \in \mathcal{F}$ , and a threshold  $t$  by Gini Impurity
9:     Create a decision tree node with  $j$ 
10:    Add two branches corresponding to  $j \leq t$  and  $j > t$ 
11:    Call CART( $\{(x_i, y_i) : (x_i, y_i) \in \mathcal{S}, x_{ij} \leq t\}, \mathcal{F} \setminus \{j\}$ )
12:    Call CART( $\{(x_i, y_i) : (x_i, y_i) \in \mathcal{S}, x_{ij} > t\}, \mathcal{F} \setminus \{j\}$ )
13:    if pruning condition is met then:
14:      return a leaf node with class  $\operatorname{argmax}_c \sum_i \mathbb{I}_{\{y_i=c\}}$ 
15:    end if
16:  end if
17:  return the decision tree
18: end function

```

CART is based on Gini Impurity. Thus, CART is well suited for large dataset with many heterogenous feature realizations.

CART branches in the continuous space by single threshold. Thus, CART yields a binary decision tree.

CART is a construction algorithm that can be used for both classification and regression tasks. It utilizes Gini impurity to determine the best feature for splitting at each node.

Breiman, L., J. H. Friedman, R. A. Olshen & C. J. Stone: CART: Classification and Regression Trees. Wadsworth: Belmont, CA, 1984.

Tree Construction Algorithms – Overview

When we construct a decision tree, we focus on the steps of **feature selection**, the selection and application of the **splitting criterion**, as well as the selection and application of the **stopping criterion**.

	ID3	C4.5	CART
Features	categorical	numerical, categorical	numerical, categorical
Target variables	categorical	categorical	numerical, categorical
Branching	one branch per category value	multi-way for categorical, binary for numeric	always binary
Pruning	No	Yes (post-pruning)	Yes (pre-pruning)
Missing values	Cannot deal with missing values	Can deal with missing values	Can deal with missing values
Splitting criterion	Information Gain	Gain Ratio	Gini Impurity (Classification) RSS (Regression)

Advantages and Disadvantages of Trees

Advantages of Decision Trees

- **Interpretability:** interpretable models, allowing human decision makers to understand how decisions are made.
- **Robustness to outliers:** DTs are relatively robust to outliers compared to other ML applications
- **Automatic feature selection:** DTs perform feature selection by determining which features provide the most informative splits
- **Little data preparation:** often no data cleaning, generation of dummy variables, or removal of blank values is necessary

Disadvantages of Decision Trees

- **Overfitting:** DTs are prone to overfitting, especially when they grow deep and complex
- **Instability:** small changes in the training dataset can result in significantly different tree structures
- **Bias towards dominant classes:** for imbalanced datasets, decision trees may show bias towards the majority class, leading to poor recall for minority classes
- **Limited expressiveness:** DTs may struggle with complex interactions between features

- Learning Decision Trees
- Pruning Decision Trees
- Popular Algorithms
- **Tree Ensembles**

Ensemble Learning

Ensemble learning combines predictions from multiple individual models $f_m(y|x)$ to a weighted combination of these individual models where the weights w_m are tunable parameters.

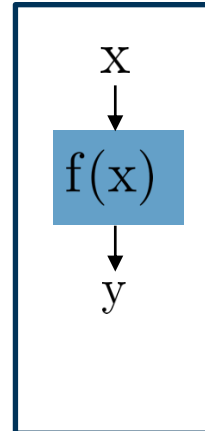
$$f(y|x) = \sum_{m \in \mathcal{M}} w_m f_m(y|x)$$

→ not limited to Decision Trees

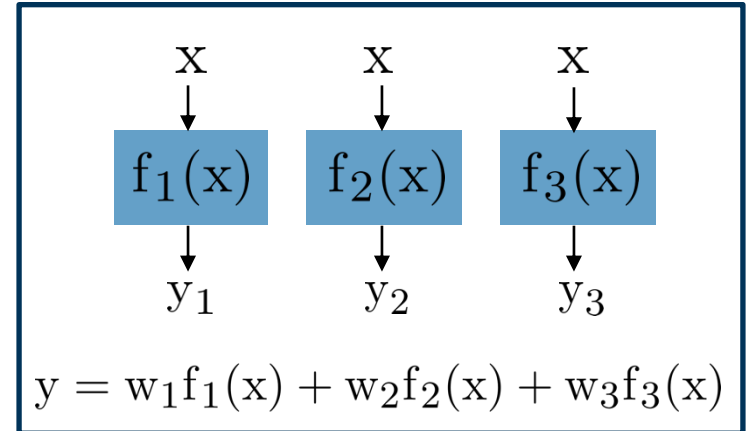
Examples for tree ensemble methods

- Bagging
- Random Forests
- Boosting
- ...

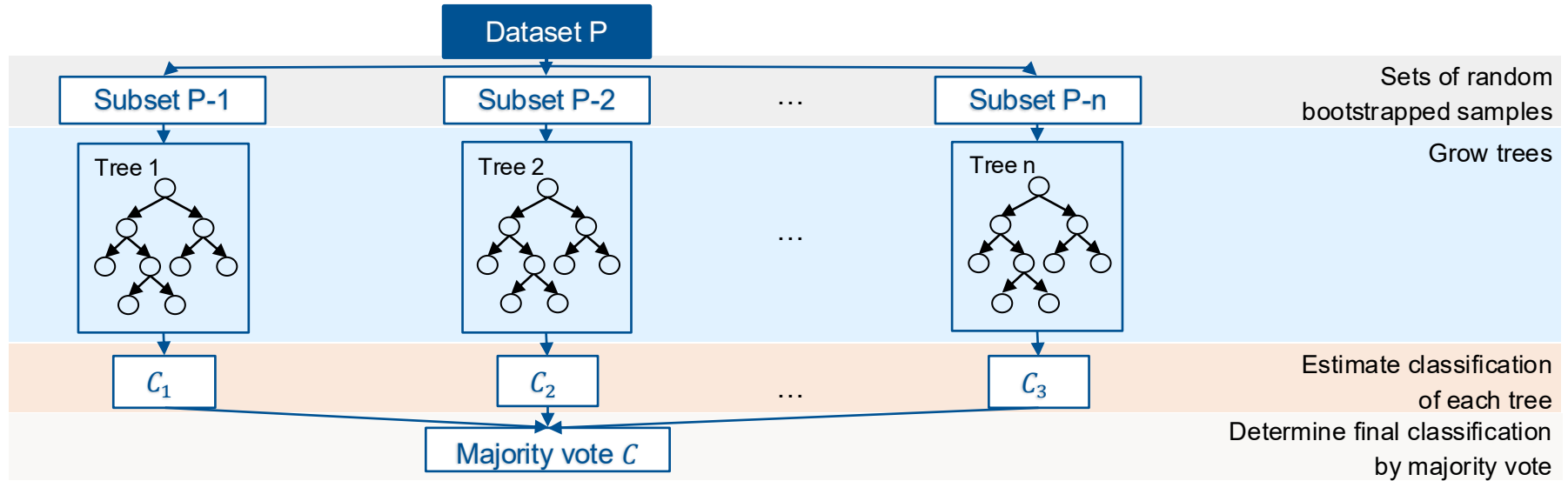
Single method



Ensemble method



Bagging (Bootstrapped Aggregating)

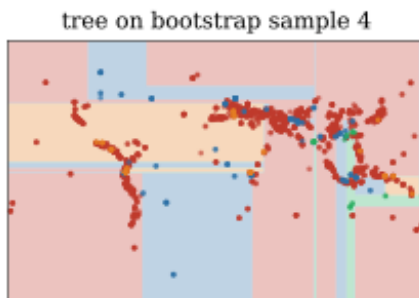
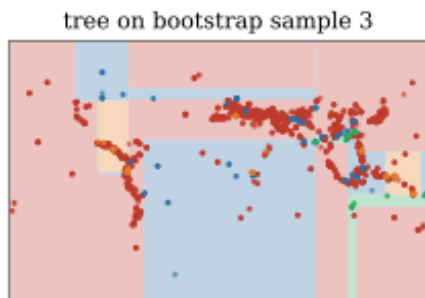
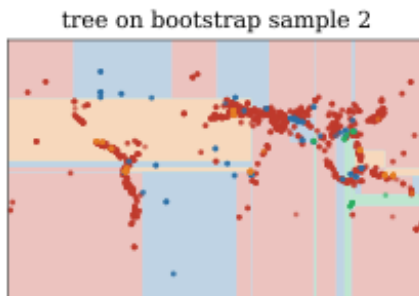
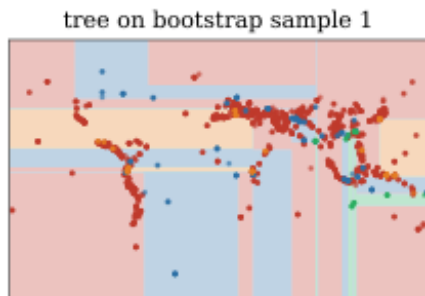


Bagging combines predictions from multiple models trained on different subsets of the data to reduce variance

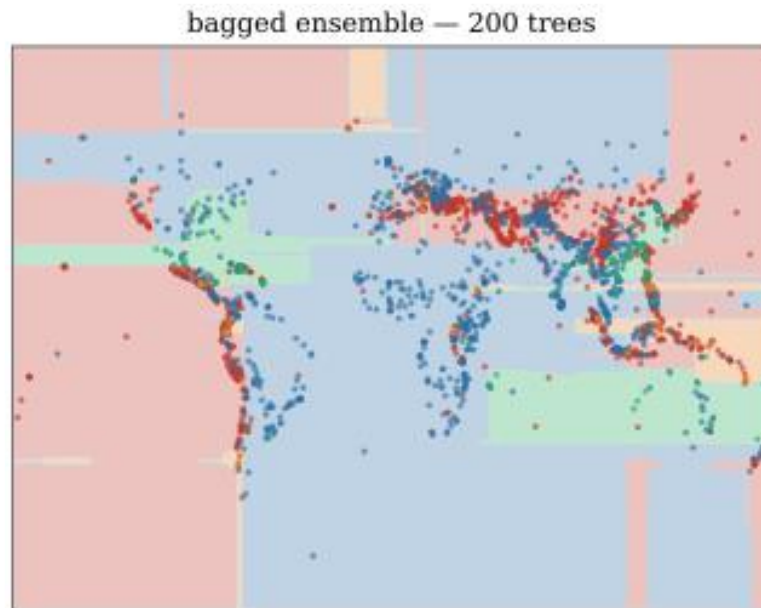
- Subsets are created via random sampling with replacement
- The final classification outcome is determined by majority vote, i.e., weights are homogenous
- Bagging aims to reduce variance → avoids overfitting

[5]

Bagging (Bootstrapped Aggregating)

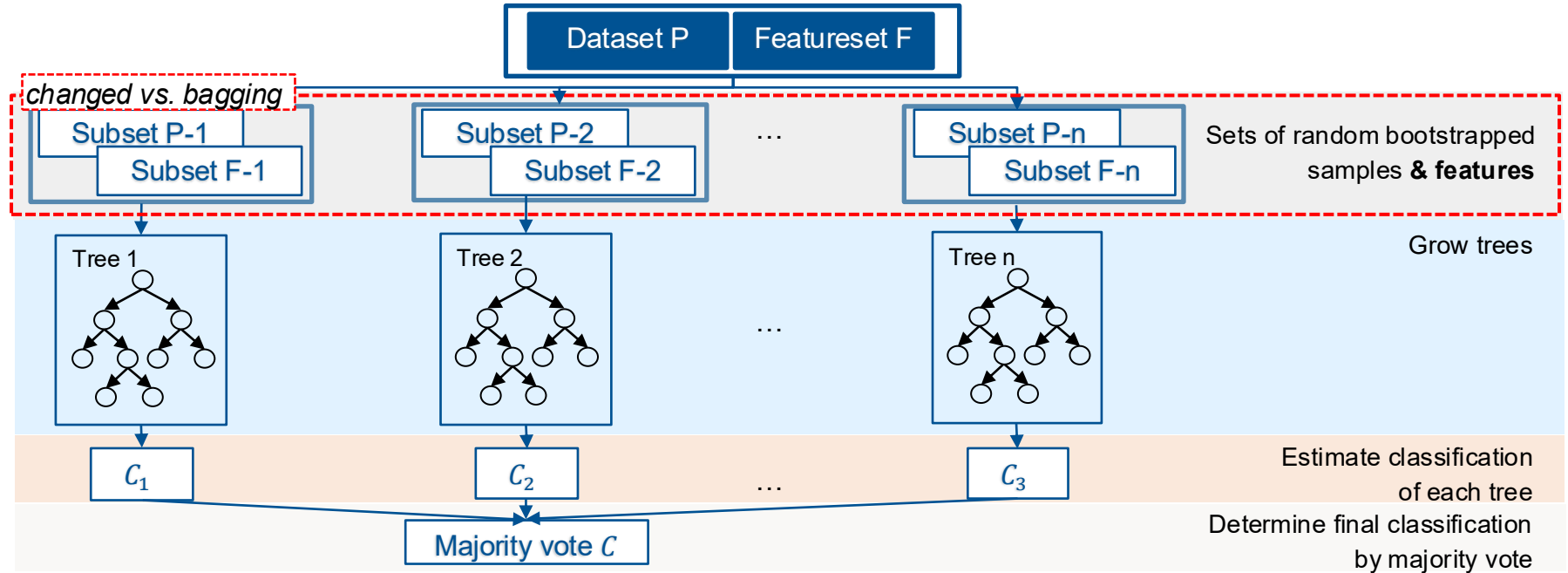


A single tree is **high-variance**: each bootstrap resample gives a very different boundary.

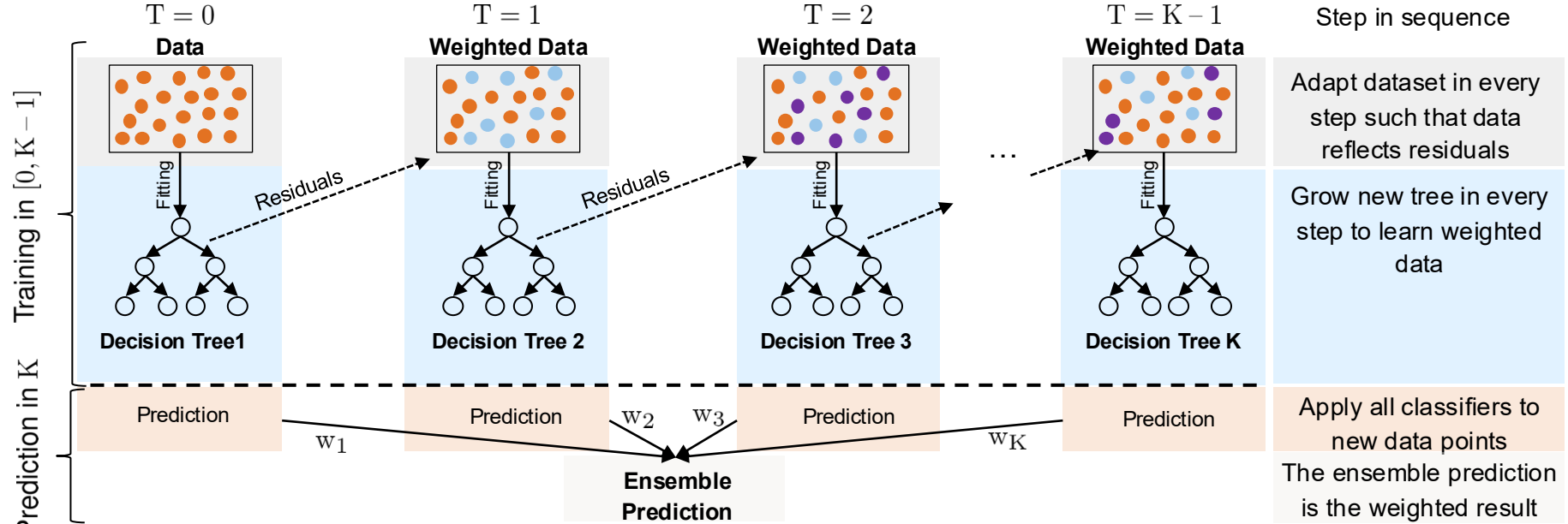


Averaging **200 trees** reduces the noise → a smooth, stable boundary (variance reduced).

Random Decision Forest

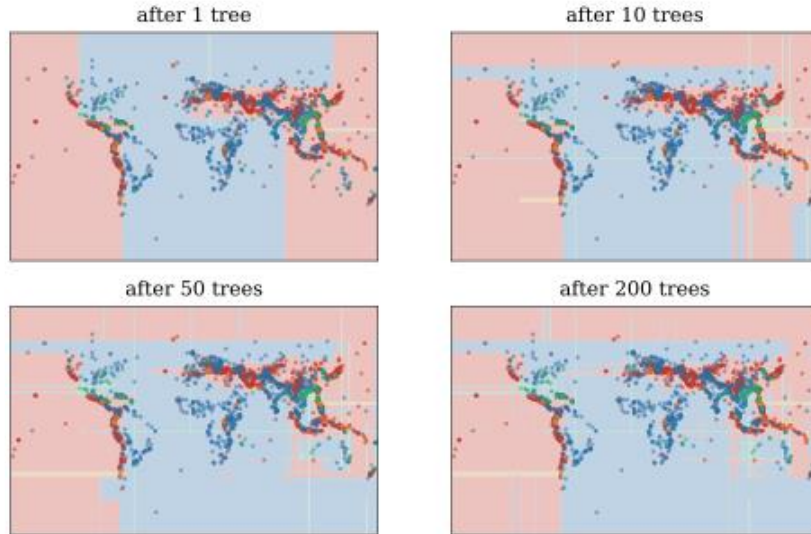


Random Forests (RFs) build multiple decision trees and aggregate their outputs to increase accuracy and control overfitting
 → By selecting both a subset of data points and a **subset of features** (features may also be sampled at each node [10] instead of using a subset of features to generate each tree [11]), RFs are less sensitive to outliers and reduce variance

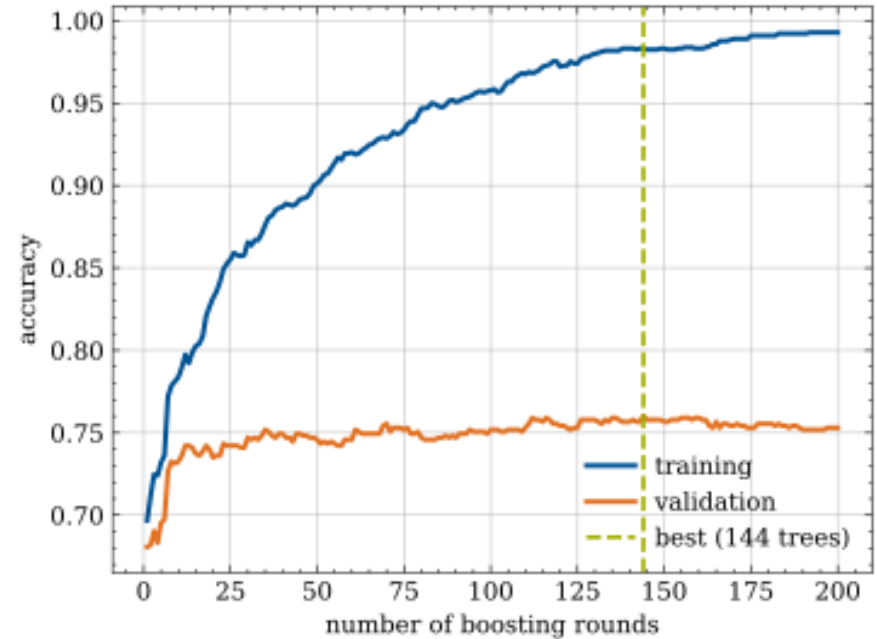


Boosting sequentially improves the accuracy of a model by predicting the residuals (e.g., misclassified points) and weighting the dataset accordingly (i.e., more focus on misclassified points). Note that these weights can be different from the weights w_m , $m \in \{1, \dots, K\}$. The main strength of boosting is the systematic reduction of bias.

Boosting



Each round adds a shallow tree that corrects the current errors — the boundary grows **richer with more rounds**.



More rounds eventually **overfit**: training $\rightarrow$ 100% while validation peaks (≈ 144) then declines.

Summary of Tree Ensemble Methods

Aspect	Bagging	Random Forest	Boosting
Model Independence	Independent	Independent	Dependent
Data Sampling	Bootstrapped samples from the training set (with replacement)	Bootstrapped samples and random subsets of features (with replacement)	Adapts weights to reduce error stepwise
Feature Selection	All features	Randomly selected subset of features	All features
Bias-Variance Trade-off	Reduces variance	Reduces variance	Reduces bias and variance
Strengths	Effective for reducing overfitting	Robust and handles high-dimensional data well due to feature randomness	Effective in improving accuracy by reducing both bias and variance

- Learning Decision Trees
- Pruning Decision Trees
- Popular Algorithms
- Tree Ensembles



What is still **unclear** from this session?